

Module 4

IEEE 1800-2005 (SystemVerilog Design Subset)

Learning objectives

- Master the first SystemVerilog standard (IEEE 1800-2005) for RTL design: logic types, always_comb/a...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["IEEE 1800-2005 Context"]

T2["logic and 2-State Types (180"]

T1 --> T2

T3["always_comb, always_ff, alwa"]

T2 --> T3

Master the first SystemVerilog standard (IEEE 1800-2005) for RTL design: logic types, always_comb/always_ff, interfaces, packages, and uniq...

Overview

- This module covers IEEE Std 1800-2005, the first SystemVerilog standard. It extends Verilog (IEEE 1...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph DUT["module4/dut"]

MUX["mux2_sv always_comb"]

CTR["counter_sv always_ff"]

DEC["decoder_unique"]

end

Module 4: DUT structure and key interfaces (see common_dut/rtl/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

TB["testbench"] --> MUX

TB --> CTR

TB --> IF

M <--> ["same bus_if()"] S

TB --> DEC

Module 4: UVM layers, agents, and checkers for this phase.

1. SystemVerilog design-subset block architecture

- ``logic``: default scalar/vector type for single-driver RTL (replaces most ``wire`/`reg`` pairs)
- ``always_comb``: explicit combinational intent; tools warn on latches and incomplete assignment
- ``always_ff``: explicit sequential intent; pairs with nonblocking updates and reset style
- Course DUTs (``module4/dut/``): ``mux2_sv.sv``, ``counter_sv.sv``, ``decoder_unique.sv``
- Packages (examples): shared ``typedef`` and functions imported where needed

2. Interface-centric connectivity architecture

- ``bus_if.sv``: bundles signals; modports ``master`` and ``slave`` define direction per role
- ``bus_master.sv`` / ``bus_slave.sv``: connect through one interface instance — fewer port lists
- Contrast with 1364: many discrete ports vs one ``interface`` port + modport view
- Reusability: same physical bus, different modport for initiator vs target

3. Case and encoding architecture

- ``unique case``: at most one branch true — synthesis-friendly decoder/FSM style
- ``priority case``: first matching branch wins — arbiter/interrupt-style priority
- ``decoder_unique.sv``: demonstrates ``unique`` + ``default`` for safe full decoding
- Migration axis: same behavior as 1364 ``case`` but with LRM-defined simulation semantics

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M4["module4.sh"] --> T1["test_mux2_sv"]

M4 --> T2["test_bus_master_slave"]

M4 --> T3["test_decoder_unique"]

T2 --> IFCHK["interface loopback check"]

Module 4: how tests, checks, and metrics close verification goals.

1. SystemVerilog testbench integration

- ``test_mux2_sv.v``, ``test_counter_sv.v``: directed tests on ``logic`/`always_comb`/`always_ff`` DUTs
- ``test_bus_master_slave.sv``: one interface instance; master drives, slave responds — loopback-style...
- ``test_decoder_unique.v``: all select values plus ``default`` branch coverage
- Tooling: Icarus/Verilator with 1800-2005 design subset; ``./scripts/module4.sh`` runs all examples

2. Interface-level verification strategy

- Stimulus at master modport; observe slave modport
(or scoreboard on interface signals)
- Connectivity test: same transaction visible on both
sides of the interface
- No UVM yet: module testbenches; classes and agents
come in later courses
- Failure modes: modport direction mismatch shows up
as compile/elab errors early

3. Semantic checks unique to 1800-2005

- ``unique case``: add vectors that would be illegal in hardware (two hot inputs) to see tool warnings
- ``always_comb``: intentionally incomplete assignment in a scratch copy to study latch warnings
- Regression: keep 1364-style equivalents from Module 3 in mind when comparing outputs in Module 7

Syllabus topics

1. IEEE 1800-2005 Context (1/2)

- logic: Single type for single-driver nets/variables (replaces wire/reg in most RTL)
- 2-state types: ``bit``, ``int``, ``byte``, etc. (mainly for testbenches; some RTL use)
- `always_comb`, `always_ff`, `always_latch`: Explicit procedural blocks with tool checks
- Interfaces and modports: Bundle signals and define direction per connection
- Packages and import: Shared types, constants, and functions across modules
- unique case, priority case: Standard semantics for case statements (no tool-specific attributes)

1. IEEE 1800-2005 Context (2/2)

- Operators: ``inside``, wildcard ``==?``, and other enhancements
- Verification: Classes, assertions (SVA), coverage, etc. (covered in later modules or other courses)
- Design/RTL: logic, `always_comb`/`always_ff`, interfaces, packages, unique/priority case
- Not in depth: Classes, SVA, coverage, constrained random (those are verification-focused)
- 1364-2005 (Verilog) is a subset of 1800-2005 (SystemVerilog).
- Valid 1364-2005 RTL is valid 1800-2005 RTL; you can migrate incrementally (e.g. `wire`→`logic`, `always`...

2. logic and 2-State Types (1800) (1/2)

- `logic`: Replaces ``wire`` and ``reg`` when there is a single driver; can be used in both continuous and...
- Rule: A ``logic`` variable must have exactly one driver (one assign, or one always block, or one port...
- 4-state: 0, 1, X, Z (same as wire/reg).
- Ports: ``input logic``, ``output logic``; no need to choose wire vs reg for single-driver ports.
- Internal: Use ``logic`` for single-driver signals; use ``wire`` only when needed (e.g. multiple drivers...
- bit: 2-state (0, 1); no X or Z. Often used in testbenches and for indices.

2. logic and 2-State Types (1800) (2/2)

- `int`, `byte`, `shortint`, `longint`: Integer types; 2-state unless declared with a 4-state base.
- Use in RTL: Optional (e.g. loop indices, parameters); many RTL designs use ``logic`` and ``integer`/`r...`

3. `always_comb`, `always_ff`, `always_latch` (1800) (1/2)

- Intent: Combinational logic; no storage; outputs are a function of inputs only.
- Sensitivity: Automatically inferred (like ``always @*``); must not contain latches.
- Tool check: Simulator/synthesis can warn if the block implies storage or is sensitive to clock edge...
- Prefer over ``always @*`` when the tool supports it: clearer intent and better checking.
- Intent: Sequential logic (flip-flops); use nonblocking assignment (``<=``) only.
- Sensitivity: Must list exactly one clock (and optionally async reset); no other event controls.

3. `always_comb`, `always_ff`, `always_latch` (1800) (2/2)

- Tool check: Can warn if blocking assignment or incomplete sensitivity is used.
- Intent: Explicit latch (level-sensitive storage).
Use sparingly; usually latches are unintended.
- When: When a latch is truly intended (e.g. transparent latch); otherwise use `always_comb` or `always_...`
- Recommendation: Prefer `always_comb` or `always_ff`; use `always_latch` only when a latch is desired.

4. Interfaces and Modports (1800)

- Signals: Declared once inside the interface; shared by all modules that use it.
- modport: Defines a view (direction) for a given role (e.g. master sees data as output, slave sees d...
- Port: Single interface port with a modport (e.g. ``bus_if.master bus``).
- Access: ``bus.signal_name``; direction is enforced by the modport.
- One interface instance: Connects master and slave; no long port lists.

5. Packages and import (1800)

- Contents: parameter, localparam, typedef, function, task, etc. (no always blocks).
- Scope: Items are in namespace ``pkg_util::`` until imported.
- `import pkg_util::*`: Import all names from the package (can cause name clashes).
- `import pkg_util::item`: Import specific items; safer when many packages are used.

6. unique case and priority case (1800)

- Semantics: At most one case item must match at runtime; if none or more than one matches, a runtime...
- Use: When the design guarantees at most one match (e.g. one-hot encoding, mutually exclusive condit...
- Synthesis: Can be implemented as parallel priority logic; no inferred priority order.
- Semantics: The first matching item has priority; if no item matches, a runtime error can be reporte...
- Use: When priority order is intended (e.g. interrupt mask).
- Synthesis: Can be implemented as a priority chain (if-then-else or similar).

7. Comparison: 1364-2005 vs 1800-2005 (Design)

- See docs/MODULE4.md

Hands-on examples

Module 4 self-check

```
$ ./scripts/module4.sh --check
```

Terminal: module4_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Exercise scaffold

```
./scripts/module4.sh --scaffold
```

Demo: logic and 2-State Types

```
$ cd module4/examples/data_types && make clean && make run
```

Terminal: ex01_data_types

```
rm -f top
iverilog -g2012 -o top logic_bit.sv
vvp top
logic (1800-2005): 4-state single driver; replaces wire/reg for RTL
  a= 10 b= 20 sum= 30 gt=0
  a= 50 b= 30 sum= 80 gt=1
logic_bit.sv:31: $finish called at 20 (1s)
```

Demo: logic Ports

```
$ cd module4/examples/logic_ports && make clean && make run
```

Terminal: ex02_logic_ports

```
rm -f top
iverilog -g2012 -o top logic_ports.v
vvp top
logic ports (1800-2005): all ports logic; no wire/reg
mux: sel=1 a=0 b=1 y=1
adder: a= 10 b= 20 sum= 30
logic_ports.v:36: $finish called at 20 (1s)
```

Demo: always_comb / always_ff

```
$ cd module4/examples/procedural && make clean && make run
```

Terminal: ex03_procedural

```
rm -f top
iverilog -g2012 -o top always_comb_ff.sv
vvp top
always_comb / always_ff (1800-2005)
  y_comb=1 z_comb=1 q=1
  y_comb=1 z_comb=1 q=1
  y_comb=1 z_comb=1 q=1
always_comb_ff.sv:50: $finish called at 45 (1s)
```

Demo: always_latch

```
$ cd module4/examples/always_latch && make clean && make run
```

```
Terminal: ex04_always_latch  
rm -f top  
iverilog -g2012 -o top always_latch.sv  
vvp top  
always_latch (1800-2005): explicit latch when en=1  
  en=0 d=1 q=x  
  en=1 d=1 q=1  
  en=1 d=0 q=0  
  en=0 d=1 q=0 (hold)  
always_latch.sv:29: $finish called at 40 (1s)
```

Demo: One Driver per logic

```
$ cd module4/examples/one_driver_logic && make clean && make run
```

Terminal: ex05_one_driver_logic

```
rm -f top
iverilog -g2012 -o top one_driver_logic.v
vvp top
One driver per logic (1800-2005): single assign, single always_ff per output
y_comb=1 q_reg=0
y_comb=1 q_reg=1
y_comb=1 q_reg=1
one_driver_logic.sv:40: $finish called at 45 (1s)
```

Demo: typedef

```
$ cd module4/examples/typedef_sv && make clean && make run
```

```
Terminal: ex06_typedef_sv  
rm -f top  
iverilog -g2012 -o top typedef_sv.sv  
vvp top  
typedef (1800-2005): byte_t = logic [7:0]  
a= 50 b= 30 sum= 80  
typedef_sv.sv:26: $finish called at 10 (1s)
```

Demo: Interfaces and Modports

```
$ cd module4/examples/interfaces && make clean && make run
```

Terminal: ex07_interfaces

```
rm -f top
iverilog -g2012 -o top bus_if_example.v
bus_if_example.v:17: syntax error
bus_if_example.v:17: Errors in port declarations.
bus_if_example.v:29: syntax error
bus_if_example.v:29: Errors in port declarations.
make: *** [Makefile:13: top] Error 4
```

Demo: Packages and import

```
$ cd module4/examples/packages && make clean && make run
```

Terminal: ex08_packages

```
rm -f top
iverilog -g2012 -o top pkg_util.v
vvp top
Package and import (1800-2005)
a= 100 b= 200 y= 300 par= 0
pkg_util.v:38: $finish called at 10 (1s)
```

Demo: Package with typedef and function

```
$ cd module4/examples/package_typedef && make clean && make run
```

```
Terminal: ex09_package_typedef  
rm -f top  
iverilog -g2012 -o top package_typedef.sv  
vvp top  
Package with typedef and function (1800-2005)  
a= 50 b= 30 sum= 80 min= 30  
a= 10 b= 90 sum=100 min= 10  
package_typedef.sv:36: $finish called at 20 (1s)
```

Demo: unique / priority case

```
$ cd module4/examples/case_unique_priority && make clean && make run
```

Terminal: ex10_case_unique_priority

```
rm -f top
iverilog -g2012 -o top decoder.sv
vvp top
unique case / priority case (1800-2005)
  decoder sel=10 y=0100
  grant req=0110 grant=0100
decoder.sv:50: $finish called at 20 (1s)
decoder.sv:26: sorry: constant selects in always_* processes are not currently supported (all bi
decoder.sv:26: sorry: constant selects in always_* processes are not currently supported (all bi
decoder.sv:26: sorry: constant selects in always_* processes are not currently supported (all bi
decoder.sv:26: sorry: constant selects in always_* processes are not currently supported (all bi
decoder.sv:12: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Demo: priority case only

```
$ cd module4/examples/priority_case && make clean && make run
```

Terminal: ex11_priority_case

```
rm -f top
iverilog -g2012 -o top priority_case.sv
vvp top
priority_case (1800-2005): first match wins
  req=0101 grant=0100 (req[2] wins)
  req=0010 grant=0010
  req=1000 grant=1000 (req[3] wins)
priority_case.sv:32: $finish called at 30 (1s)
priority_case.sv:10: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:10: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:10: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:10: sorry: constant selects in always_* processes are not currently supported (
```

Demo: Migration from 1364

```
$ cd module4/examples/migration && make clean && make run
```

Terminal: ex12_migration

```
rm -f top
iverilog -g2012 -o top migration.sv
vvp top
Migration (1800-2005): logic + always_comb (was wire/reg + always @*)
  sel=0 a=0 b=1 y=0
  sel=1 a=0 b=1 y=1
migration.sv:26: $finish called at 20 (1s)
```

Practice & assessment

Exercises

- logic and always_comb/always_ff
- Interface
- Package
- unique case
- Migration Summary

Exercises

- Multiple Drivers on logic
- Using always_comb with Latched Behavior
- Wrong Modport or Direction
- Forgetting to import Package
- unique case with Multiple Matches

Summary & next steps

- Master the first SystemVerilog standard (IEEE 1800-2005) for RTL design: logic types, always_comb/a...
- Complete module4/CHECKLIST.md
- Run ./scripts/module4.sh --check
- Next: Next module in course